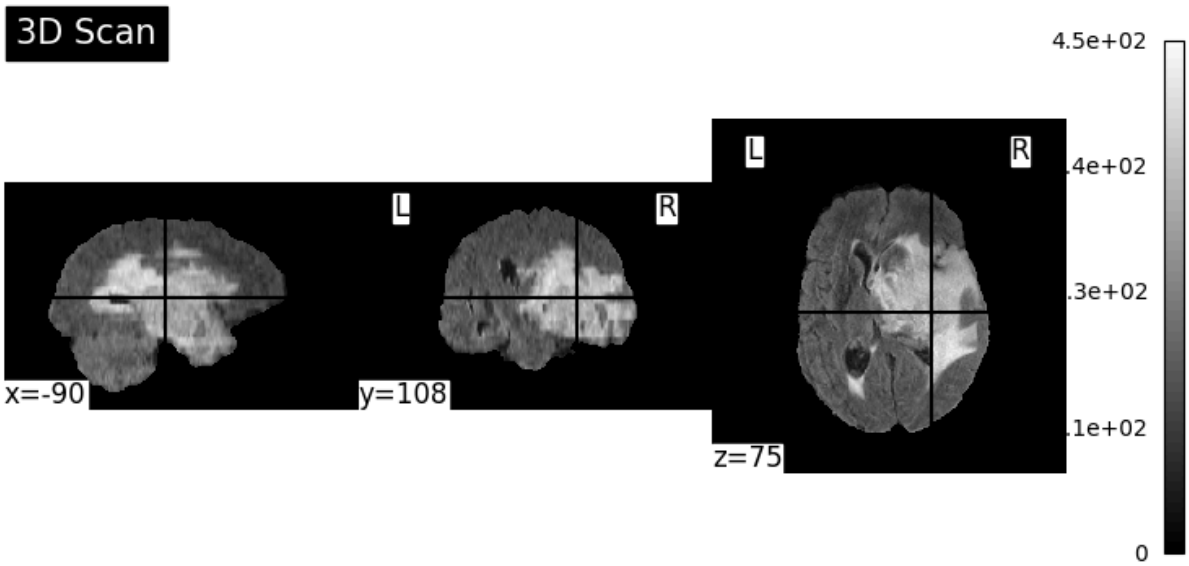
In [1]:
```
from preprocess.conversion import visualise_datapoint, visualise_scan_data, convert
```

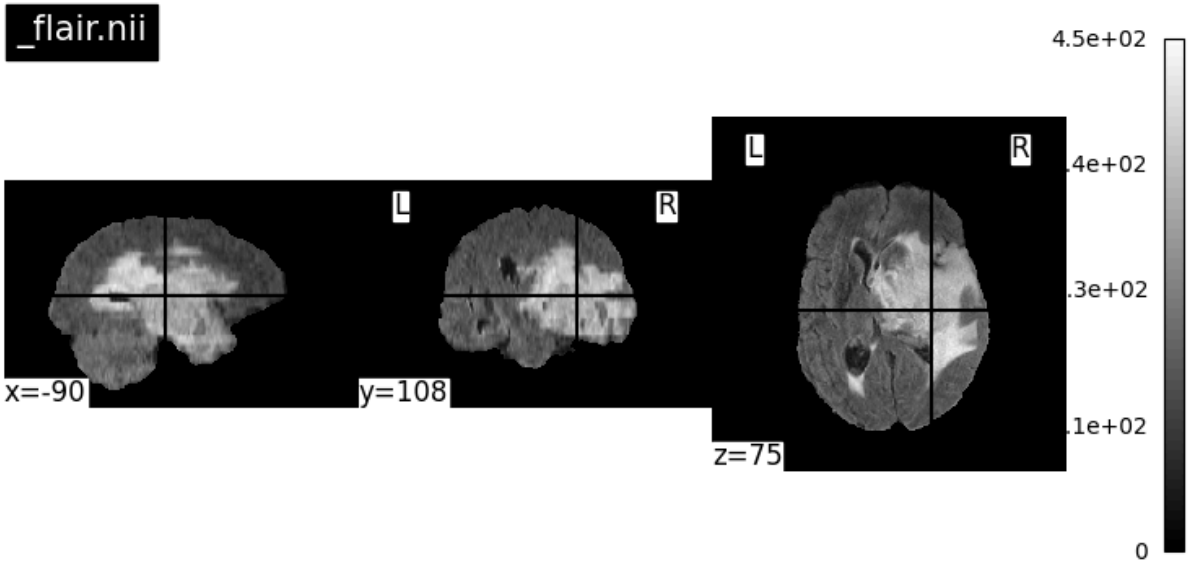
To visualise the samples, I have created several helper functions in preprocess/conversion.py. We can visualise a single datapoint by calling visualise_scan_data(). We can also visualise all four, by calling visualise_datapoint().

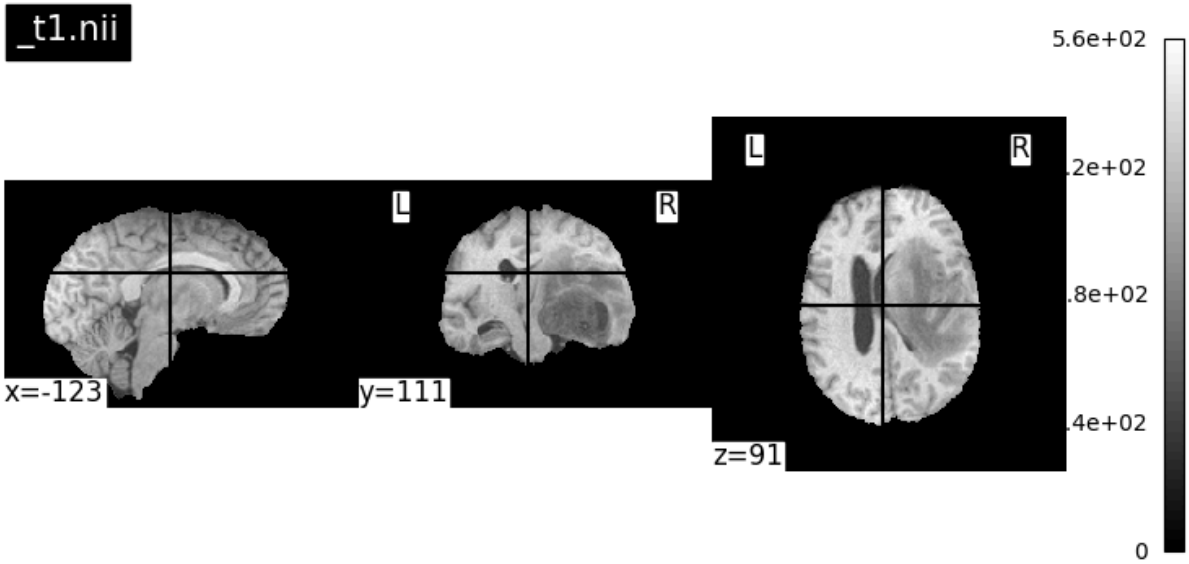
In [2]:
```
datapoints = []

datapoints.append("\BraTS20_Training_001\BraTS20_Training_001_flair.nii")
datapoints.append("\BraTS20_Training_001\BraTS20_Training_001_seg.nii")

for datapoint in datapoints:
    visualise_scan_data(datapoint, is_train=True)
```

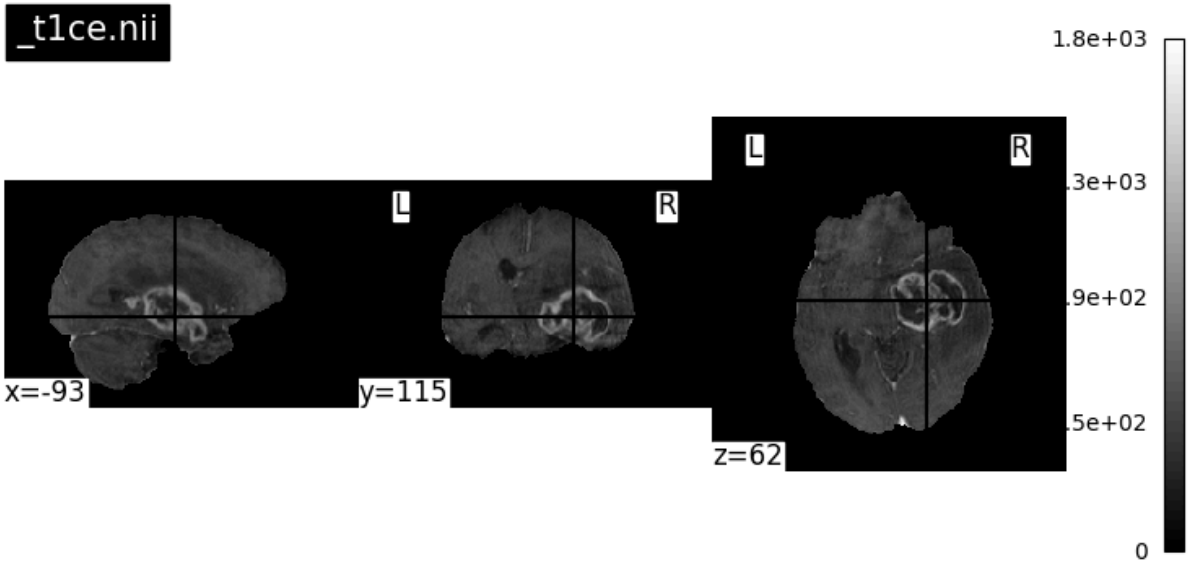


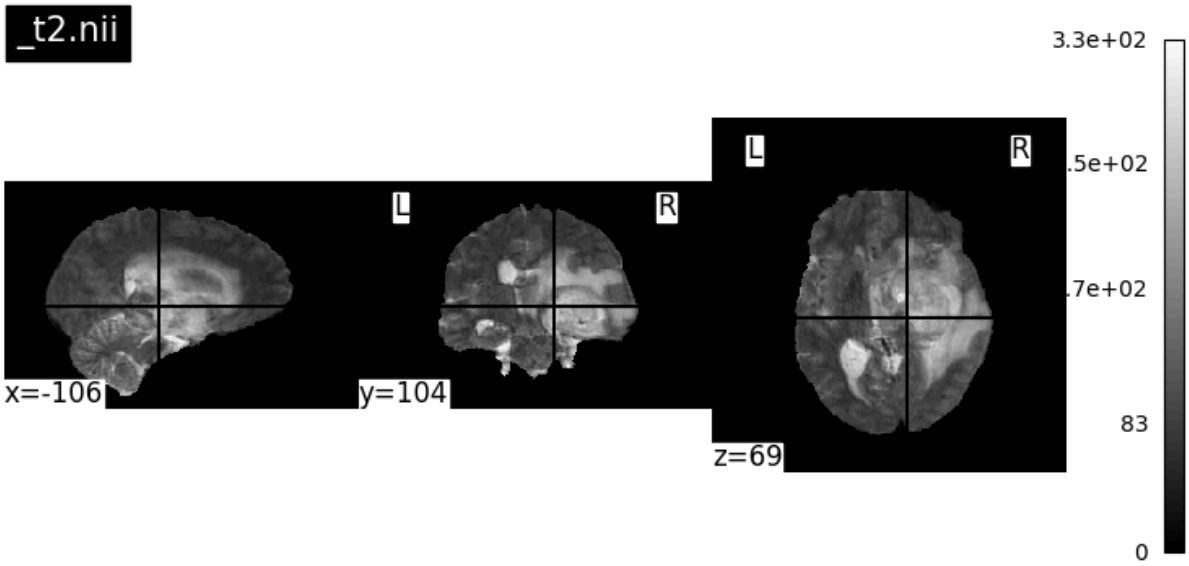
To visualise all four parts, we can use:

In [3]:
```
path = "\BraTS20_Training_001"
visualise_datapoint(path, is_train=True)
```




_t1.nii
L
R
x=-123
y=111
z=91
5.6e+02
.2e+02
.8e+02
.4e+02
0


_t1ce.nii
L
R
x=-93
y=115
z=62
1.8e+03
.3e+03
.9e+02
.5e+02
0


_t2.nii
L
R
x=-106
y=104
z=69
3.3e+02
.5e+02
.7e+02
83
0

Before training the model, all .nii.gz files should be converted into NumPy arrays and saved locally to significantly reduce I/O overhead during training, since loading compressed medical imaging files repeatedly is slow. As part of this conversion process, each scan must be preprocessed to ensure consistency across the dataset: first, remove empty background regions by cropping to the smallest bounding box containing meaningful information, which reduces memory usage and computational cost. then, we apply scaling using an appropriate scaler, in this case we used z-score normalisation to normalize intensities and stabilize model training.

In [2]:
```
convert_all_npy()
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[2], line 1
----> 1 convert_all_npy()

NameError: name 'convert_all_npy' is not defined
```

Now that we have completed the converting all the files into numpy format, we have to split the dataset into a training set and a testing set. If we look above, we can see that the t1 image does not look like it provides much useful information. Therefore, it will be skipped and not added into our training pipeline. Now we need to load our data into main memory.

In [1]:
```
from preprocess.splitting import train_test_split, combine_from_npy

features, labels = combine_from_npy()
```

In [2]:
```
features.shape
```

Out[2]:
```
(220, 3, 128, 128, 128)
```

In [3]:
```
labels.shape
```

Out[3]:
```
(220, 4, 128, 128, 128)
```

In [4]:
```
import torch
if torch.cuda.is_available():
    device = torch.device('cuda')
    print("Using cuda")
else:
    device = torch.device('cpu')
    print("Using cpu")
```

```
Using cuda
```

In [5]:
```
train_dataloader, test_dataloader = train_test_split(features, labels, device)
del features
del labels
```

```
Training batches: 88
Testing batches: 22
```

In [6]:
```
from models.UNet.unet import UNet
from models.UNet.params import UNetParams

params = UNetParams()
model = UNet(params=params).to(device=device)
```

In [7]:
```
from utility.utils import init_weights

model.apply(init_weights)
```

Out[7]:
```
UNet(
  (down_conv1): ConvReLUBlock(
    (conv3d_1): Conv3d(3, 16, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=(1,
1, 1))
    (conv3d_2): Conv3d(16, 16, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=
(1, 1, 1))
    (relu1): LeakyReLU(negative_slope=0.01, inplace=True)
    (relu2): LeakyReLU(negative_slope=0.01, inplace=True)
    (grp_norm): GroupNorm(8, 16, eps=1e-05, affine=True)
  )
  (down_conv2): ConvReLUBlock(
    (conv3d_1): Conv3d(16, 32, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=
(1, 1, 1))
    (conv3d_2): Conv3d(32, 32, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=
(1, 1, 1))
    (relu1): LeakyReLU(negative_slope=0.01, inplace=True)
    (relu2): LeakyReLU(negative_slope=0.01, inplace=True)
    (grp_norm): GroupNorm(8, 32, eps=1e-05, affine=True)
  )
  (down_conv3): ConvReLUBlock(
    (conv3d_1): Conv3d(32, 64, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=
(1, 1, 1))
    (conv3d_2): Conv3d(64, 64, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=
(1, 1, 1))
    (relu1): LeakyReLU(negative_slope=0.01, inplace=True)
    (relu2): LeakyReLU(negative_slope=0.01, inplace=True)
    (grp_norm): GroupNorm(8, 64, eps=1e-05, affine=True)
  )
  (down_conv4): ConvReLUBlock(
    (conv3d_1): Conv3d(64, 128, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=
(1, 1, 1))
    (conv3d_2): Conv3d(128, 128, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=
(1, 1, 1))
    (relu1): LeakyReLU(negative_slope=0.01, inplace=True)
    (relu2): LeakyReLU(negative_slope=0.01, inplace=True)
    (grp_norm): GroupNorm(8, 128, eps=1e-05, affine=True)
  )
  (bottlenck): ConvReLUBlock(
    (conv3d_1): Conv3d(128, 256, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=
(1, 1, 1))
    (conv3d_2): Conv3d(256, 256, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=
(1, 1, 1))
    (relu1): LeakyReLU(negative_slope=0.01, inplace=True)
    (relu2): LeakyReLU(negative_slope=0.01, inplace=True)
    (grp_norm): GroupNorm(8, 256, eps=1e-05, affine=True)
  )
  (up_conv1): TransposedConvBlock(
    (conv3d_transposed): ConvTranspose3d(256, 128, kernel_size=(2, 2, 2), stride=
(2, 2, 2))
    (conv3d_block): ConvReLUBlock(
      (conv3d_1): Conv3d(256, 128, kernel_size=(3, 3, 3), stride=(1, 1, 1), paddin
g=(1, 1, 1))
      (conv3d_2): Conv3d(128, 128, kernel_size=(3, 3, 3), stride=(1, 1, 1), paddin
g=(1, 1, 1))
      (relu1): LeakyReLU(negative_slope=0.01, inplace=True)
      (relu2): LeakyReLU(negative_slope=0.01, inplace=True)
```

```
      (grp_norm): GroupNorm(8, 128, eps=1e-05, affine=True)
    )
    (groupnorm): GroupNorm(8, 256, eps=1e-05, affine=False)
  )
  (up_conv2): TransposedConvBlock(
    (conv3d_transposed): ConvTranspose3d(128, 64, kernel_size=(2, 2, 2), stride=
(2, 2, 2))
    (conv3d_block): ConvReLUBlock(
      (conv3d_1): Conv3d(128, 64, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding
=(1, 1, 1))
      (conv3d_2): Conv3d(64, 64, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=
(1, 1, 1))
      (relu1): LeakyReLU(negative_slope=0.01, inplace=True)
      (relu2): LeakyReLU(negative_slope=0.01, inplace=True)
      (grp_norm): GroupNorm(8, 64, eps=1e-05, affine=True)
    )
    (groupnorm): GroupNorm(8, 128, eps=1e-05, affine=False)
  )
  (up_conv3): TransposedConvBlock(
    (conv3d_transposed): ConvTranspose3d(64, 32, kernel_size=(2, 2, 2), stride=(2,
2, 2))
    (conv3d_block): ConvReLUBlock(
      (conv3d_1): Conv3d(64, 32, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=
(1, 1, 1))
      (conv3d_2): Conv3d(32, 32, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=
(1, 1, 1))
      (relu1): LeakyReLU(negative_slope=0.01, inplace=True)
      (relu2): LeakyReLU(negative_slope=0.01, inplace=True)
      (grp_norm): GroupNorm(8, 32, eps=1e-05, affine=True)
    )
    (groupnorm): GroupNorm(8, 64, eps=1e-05, affine=False)
  )
  (up_conv4): TransposedConvBlock(
    (conv3d_transposed): ConvTranspose3d(32, 16, kernel_size=(2, 2, 2), stride=(2,
2, 2))
    (conv3d_block): ConvReLUBlock(
      (conv3d_1): Conv3d(32, 16, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=
(1, 1, 1))
      (conv3d_2): Conv3d(16, 16, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=
(1, 1, 1))
      (relu1): LeakyReLU(negative_slope=0.01, inplace=True)
      (relu2): LeakyReLU(negative_slope=0.01, inplace=True)
      (grp_norm): GroupNorm(8, 16, eps=1e-05, affine=True)
    )
    (groupnorm): GroupNorm(8, 32, eps=1e-05, affine=False)
  )
  (maxpool): MaxPool3d(kernel_size=(2, 2, 2), stride=(2, 2, 2), padding=0, dilatio
n=1, ceil_mode=False)
  (softmax): Softmax(dim=1)
  (final_conv): Conv3d(16, 4, kernel_size=(1, 1, 1), stride=(1, 1, 1))
)
```

In [7]:

```python
import torch._dynamo
print(torch._dynamo.list_backends())
```

```
['cudagraphs', 'inductor', 'openxla', 'tvm']
```

In [8]:
```
model = torch.compile(model=model, backend="inductor")
```

In [9]:
```
from tqdm import tqdm

from utility.utils import train_loop, inference_loop
from utility.metrics import DiceCELoss
epochs = 40

torch.backends.cudnn.benchmark = True
torch.set_float32_matmul_precision("high")

train_losses = []
test_losses = []
train_dice_scores = []
test_dice_scores = []

optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer,
    mode='min',
    factor=0.5,
    patience=10,
    min_lr=1e-6
)
loss = DiceCELoss().to(device=device)

for epoch in tqdm(range(1, epochs+1)):

    train_epoch_loss, train_dice_score = train_loop(
        model=model,
        train_dataloader=train_dataloader,
        loss_fn=loss,
        optimizer=optimizer,
        device=device
    )
    test_epoch_loss, test_dice_score = inference_loop(
        model=model,
        test_dataloader=test_dataloader,
        loss_fn=loss,
        device=device
    )

    print(f"training loss : {train_epoch_loss}")
    print(f"testing loss : {test_epoch_loss}")

    train_losses.append(train_epoch_loss)
    test_losses.append(test_epoch_loss)
    train_dice_scores.append(train_dice_score)
    test_dice_scores.append(test_dice_score)
```

```
  0%|          | 0/40 [00:00<?, ?it/s]
Training Loop: 88it [05:08,  3.50s/it]
Evaluation Loop: 22it [00:09,  2.26it/s]
  2%|▌          | 1/40 [05:21<3:28:46, 321.18s/it]
```

```
training loss : 2.098281368613243
testing loss : 1.970611723986539
Training Loop: 88it [05:01,  3.42s/it]
Evaluation Loop: 22it [00:08,  2.66it/s]
  5%|▌          | 2/40 [10:30<3:19:09, 314.45s/it]
training loss : 1.9461491338231347
testing loss : 1.8990217338908801
Training Loop: 88it [05:02,  3.44s/it]
Evaluation Loop: 22it [00:08,  2.68it/s]
  8%|▉          | 3/40 [15:42<3:12:58, 312.94s/it]
training loss : 1.847719443115321
testing loss : 1.777408236807043
Training Loop: 88it [05:00,  3.42s/it]
Evaluation Loop: 22it [00:08,  2.71it/s]
 10%|▉          | 4/40 [20:51<3:06:50, 311.41s/it]
training loss : 1.7326985665343024
testing loss : 1.6828903880986301
Training Loop: 88it [05:03,  3.45s/it]
Evaluation Loop: 22it [00:08,  2.65it/s]
 12%|▉          | 5/40 [26:03<3:01:45, 311.59s/it]
training loss : 1.6584257131273097
testing loss : 1.6550857587294145
Training Loop: 88it [05:01,  3.42s/it]
Evaluation Loop: 22it [00:08,  2.67it/s]
 15%|▉          | 6/40 [31:12<2:56:11, 310.92s/it]
training loss : 1.6086854934692383
testing loss : 1.6751895438541065
Training Loop: 88it [05:09,  3.52s/it]
Evaluation Loop: 22it [00:08,  2.67it/s]
 18%|▉          | 7/40 [36:30<2:52:19, 313.30s/it]
training loss : 1.596492732113058
testing loss : 1.6626846627755598
Training Loop: 88it [05:02,  3.44s/it]
Evaluation Loop: 22it [00:08,  2.64it/s]
 20%|▉          | 8/40 [41:42<2:46:43, 312.62s/it]
training loss : 1.5686777220530943
testing loss : 1.554041943766854
Training Loop: 88it [05:03,  3.45s/it]
Evaluation Loop: 22it [00:08,  2.68it/s]
 22%|▉          | 9/40 [46:53<2:41:21, 312.31s/it]
training loss : 1.5322751646692103
testing loss : 1.5230123888362537
Training Loop: 88it [05:07,  3.49s/it]
Evaluation Loop: 22it [00:07,  2.75it/s]
 25%|▉          | 10/40 [52:09<2:36:38, 313.28s/it]
training loss : 1.4959698739376934
testing loss : 1.523072204806588
Training Loop: 88it [05:07,  3.49s/it]
Evaluation Loop: 22it [00:08,  2.66it/s]
 28%|▉          | 11/40 [57:24<2:31:46, 314.01s/it]
training loss : 1.483271611007777
testing loss : 1.4694022102789446
```

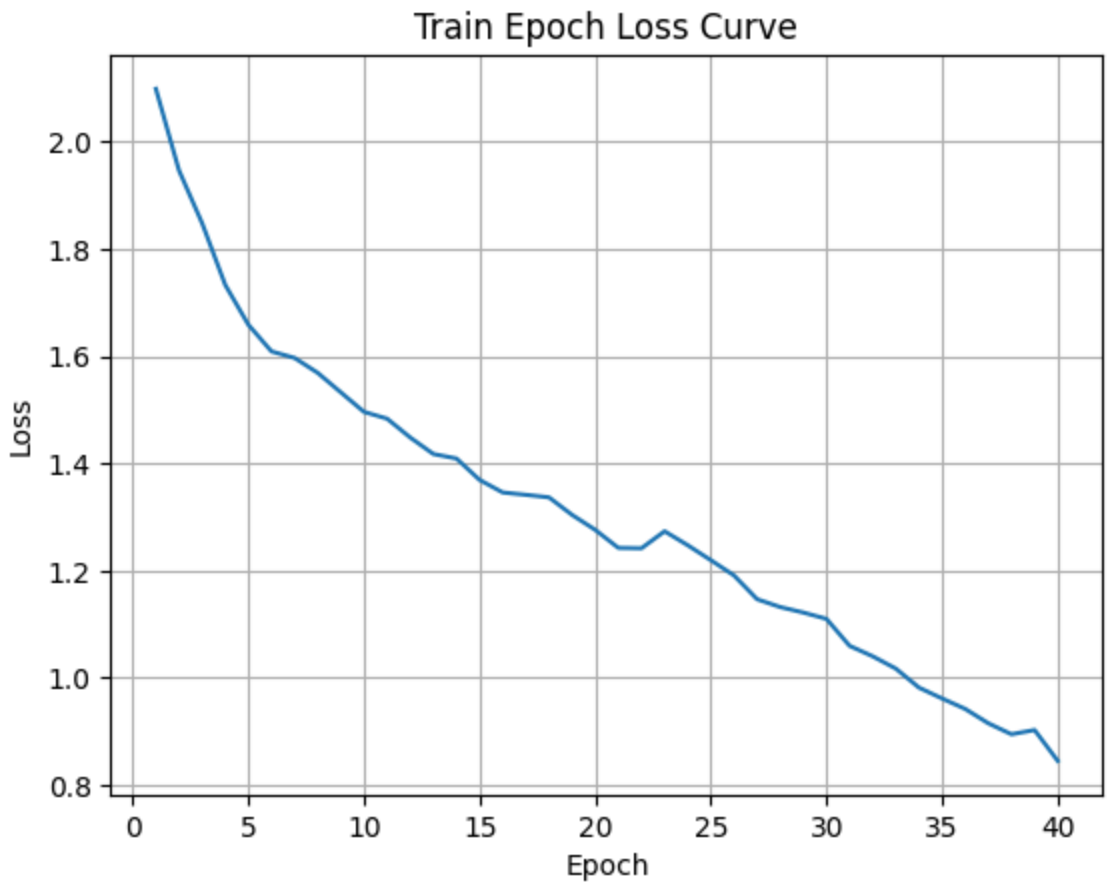
```
Training Loop: 88it [05:09,  3.51s/it]
Evaluation Loop: 22it [00:08,  2.68it/s]
 30%|└└└        | 12/40 [1:02:42<2:27:01, 315.05s/it]
training loss : 1.4481329389593818
testing loss : 1.4617286974733525
Training Loop: 88it [05:08,  3.50s/it]
Evaluation Loop: 22it [00:08,  2.68it/s]
 32%|└└└        | 13/40 [1:07:58<2:21:58, 315.52s/it]
training loss : 1.4173797301270745
testing loss : 1.4636004350402139
Training Loop: 88it [05:09,  3.52s/it]
Evaluation Loop: 22it [00:08,  2.66it/s]
 35%|└└└        | 14/40 [1:13:16<2:17:01, 316.20s/it]
training loss : 1.4089892167936673
testing loss : 1.4069985855709424
Training Loop: 88it [05:05,  3.48s/it]
Evaluation Loop: 22it [00:08,  2.64it/s]
 38%|└└└└       | 15/40 [1:18:30<2:11:30, 315.61s/it]
training loss : 1.3690900111740285
testing loss : 1.3694874373349277
Training Loop: 88it [05:06,  3.48s/it]
Evaluation Loop: 22it [00:08,  2.74it/s]
 40%|└└└└       | 16/40 [1:23:45<2:06:07, 315.30s/it]
training loss : 1.345671143044125
testing loss : 1.3842862410978838
Training Loop: 88it [05:06,  3.48s/it]
Evaluation Loop: 22it [00:08,  2.72it/s]
 42%|└└└└└      | 17/40 [1:28:59<2:00:45, 315.02s/it]
training loss : 1.3412890068509362
testing loss : 1.3842406218702143
Training Loop: 88it [05:07,  3.49s/it]
Evaluation Loop: 22it [00:08,  2.71it/s]
 45%|└└└└└      | 18/40 [1:34:15<1:55:33, 315.17s/it]
training loss : 1.336575211449103
testing loss : 1.3548769625750454
Training Loop: 88it [05:10,  3.53s/it]
Evaluation Loop: 22it [00:08,  2.47it/s]
 48%|└└└└└└     | 19/40 [1:39:34<1:50:45, 316.45s/it]
training loss : 1.3038550872694363
testing loss : 1.3098744424906643
Training Loop: 88it [05:57,  4.06s/it]
Evaluation Loop: 22it [00:08,  2.48it/s]
 50%|└└└└└└     | 20/40 [1:45:41<1:50:30, 331.53s/it]
training loss : 1.2760114127939397
testing loss : 1.3024665225635876
Training Loop: 88it [06:00,  4.10s/it]
Evaluation Loop: 22it [00:08,  2.47it/s]
 52%|└└└└└└└    | 21/40 [1:51:51<1:48:37, 343.01s/it]
training loss : 1.2423167418349872
testing loss : 1.2592520171945745
Training Loop: 88it [05:58,  4.08s/it]
Evaluation Loop: 22it [00:09,  2.42it/s]
 55%|└└└└└└└    | 22/40 [1:57:59<1:45:09, 350.53s/it]
```

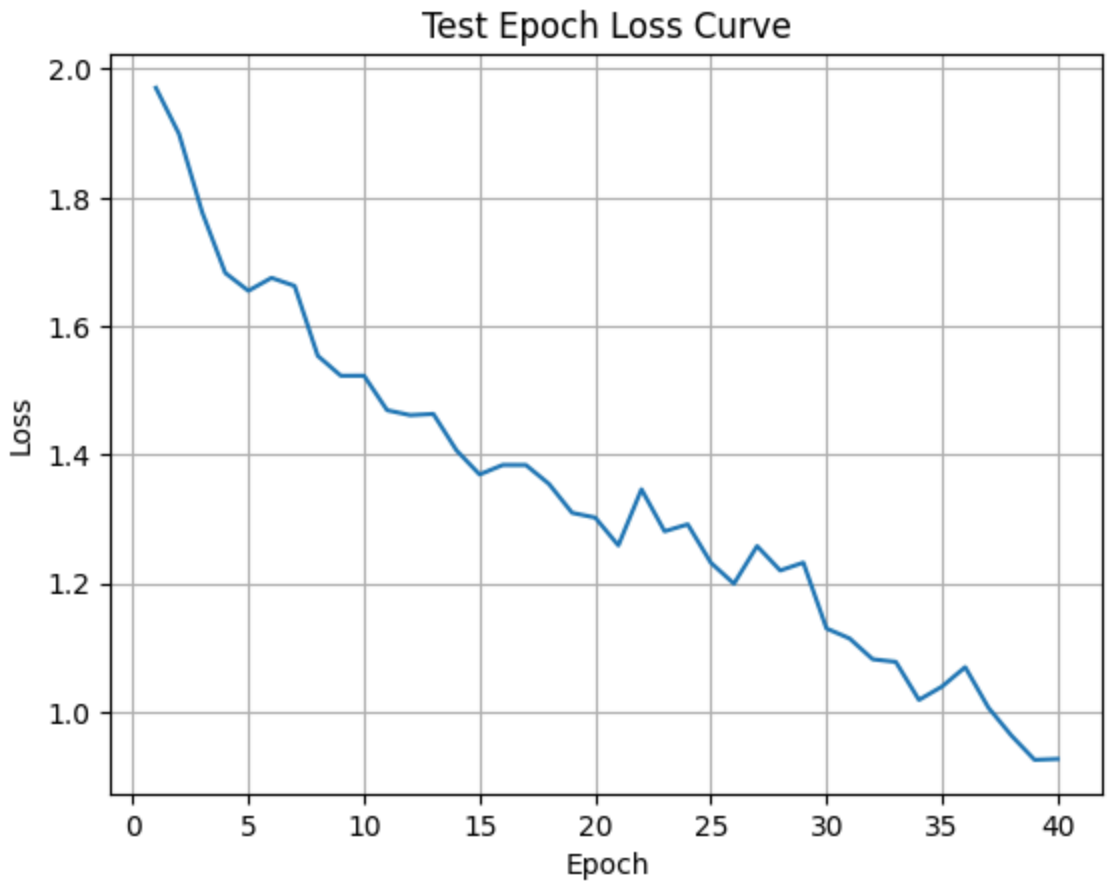
```
training loss : 1.2416283813389866
testing loss : 1.3464253544807434
Training Loop: 88it [06:00,  4.09s/it]
Evaluation Loop: 22it [00:08,  2.46it/s]
 57%|▊▊▊▊▊▊     | 23/40 [2:04:08<1:40:54, 356.12s/it]
training loss : 1.2733771272680976
testing loss : 1.2811659953810952
Training Loop: 88it [05:59,  4.08s/it]
Evaluation Loop: 22it [00:08,  2.49it/s]
 60%|▊▊▊▊▊▊     | 24/40 [2:10:16<1:35:56, 359.80s/it]
training loss : 1.2472816834395581
testing loss : 1.2919736287810586
Training Loop: 88it [05:54,  4.02s/it]
Evaluation Loop: 22it [00:08,  2.49it/s]
 62%|▊▊▊▊▊▊▊    | 25/40 [2:16:19<1:30:11, 360.77s/it]
training loss : 1.2194461199370297
testing loss : 1.2323219803246586
Training Loop: 88it [05:54,  4.03s/it]
Evaluation Loop: 22it [00:08,  2.48it/s]
 65%|▊▊▊▊▊▊▊    | 26/40 [2:22:23<1:24:23, 361.66s/it]
training loss : 1.1910384256731381
testing loss : 1.1997094235636971
Training Loop: 88it [05:47,  3.95s/it]
Evaluation Loop: 22it [00:08,  2.63it/s]
 68%|▊▊▊▊▊▊▊▊   | 27/40 [2:28:19<1:18:00, 360.04s/it]
training loss : 1.1466550081968307
testing loss : 1.2582298761064357
Training Loop: 88it [05:13,  3.56s/it]
Evaluation Loop: 22it [00:08,  2.62it/s]
 70%|▊▊▊▊▊▊▊▊   | 28/40 [2:33:41<1:09:43, 348.67s/it]
training loss : 1.1320401430130005
testing loss : 1.220284635370428
Training Loop: 88it [05:09,  3.51s/it]
Evaluation Loop: 22it [00:08,  2.70it/s]
 72%|▊▊▊▊▊▊▊▊   | 29/40 [2:38:59<1:02:11, 339.25s/it]
training loss : 1.1217996816743503
testing loss : 1.2324450774626299
Training Loop: 88it [05:08,  3.50s/it]
Evaluation Loop: 22it [00:08,  2.70it/s]
 75%|▊▊▊▊▊▊▊▊   | 30/40 [2:44:15<55:23, 332.38s/it]
training loss : 1.1099941832098095
testing loss : 1.1301498927853324
Training Loop: 88it [05:09,  3.52s/it]
Evaluation Loop: 22it [00:08,  2.74it/s]
 78%|▊▊▊▊▊▊▊▊▊  | 31/40 [2:49:33<49:12, 328.06s/it]
training loss : 1.0597248578613454
testing loss : 1.1145555593750693
Training Loop: 88it [05:07,  3.50s/it]
Evaluation Loop: 22it [00:08,  2.71it/s]
 80%|▊▊▊▊▊▊▊▊   | 32/40 [2:54:49<43:15, 324.39s/it]
training loss : 1.040198196064342
testing loss : 1.0822223397818478
```

```
Training Loop: 88it [05:04,  3.46s/it]
Evaluation Loop: 22it [00:08,  2.72it/s]
 82%|▊▊▊▊▊▊▊▊▊▊  | 33/40 [3:00:01<37:25, 320.81s/it]
training loss : 1.0172631916674701
testing loss : 1.078158682042902
Training Loop: 88it [05:05,  3.48s/it]
Evaluation Loop: 22it [00:08,  2.69it/s]
 85%|▊▊▊▊▊▊▊▊▊▊  | 34/40 [3:05:15<31:52, 318.80s/it]
training loss : 0.9818693135272373
testing loss : 1.0190077315677295
Training Loop: 88it [05:04,  3.46s/it]
Evaluation Loop: 22it [00:08,  2.68it/s]
 88%|▊▊▊▊▊▊▊▊▊▊  | 35/40 [3:10:28<26:24, 316.96s/it]
training loss : 0.9615684639323842
testing loss : 1.039881714365699
Training Loop: 88it [05:07,  3.50s/it]
Evaluation Loop: 22it [00:08,  2.64it/s]
 90%|▊▊▊▊▊▊▊▊▊▊  | 36/40 [3:15:44<21:06, 316.72s/it]
training loss : 0.9421049065210603
testing loss : 1.0699753273617139
Training Loop: 88it [05:22,  3.67s/it]
Evaluation Loop: 22it [00:08,  2.52it/s]
 92%|▊▊▊▊▊▊▊▊▊▊  | 37/40 [3:21:16<16:03, 321.20s/it]
training loss : 0.9153361178257249
testing loss : 1.0071789568120784
Training Loop: 88it [06:01,  4.10s/it]
Evaluation Loop: 22it [00:08,  2.53it/s]
 95%|▊▊▊▊▊▊▊▊▊▊  | 38/40 [3:27:26<11:11, 335.77s/it]
training loss : 0.8948437002572146
testing loss : 0.9635659645904194
Training Loop: 88it [06:00,  4.10s/it]
Evaluation Loop: 22it [00:08,  2.45it/s]
 98%|▊▊▊▊▊▊▊▊▊▊  | 39/40 [3:33:35<05:45, 345.98s/it]
training loss : 0.9027705761519346
testing loss : 0.9258584000847556
Training Loop: 88it [06:00,  4.10s/it]
Evaluation Loop: 22it [00:08,  2.48it/s]
100%|▊▊▊▊▊▊▊▊▊▊| 40/40 [3:39:45<00:00, 329.65s/it]
training loss : 0.8453123725273393
testing loss : 0.9272399246692657
```

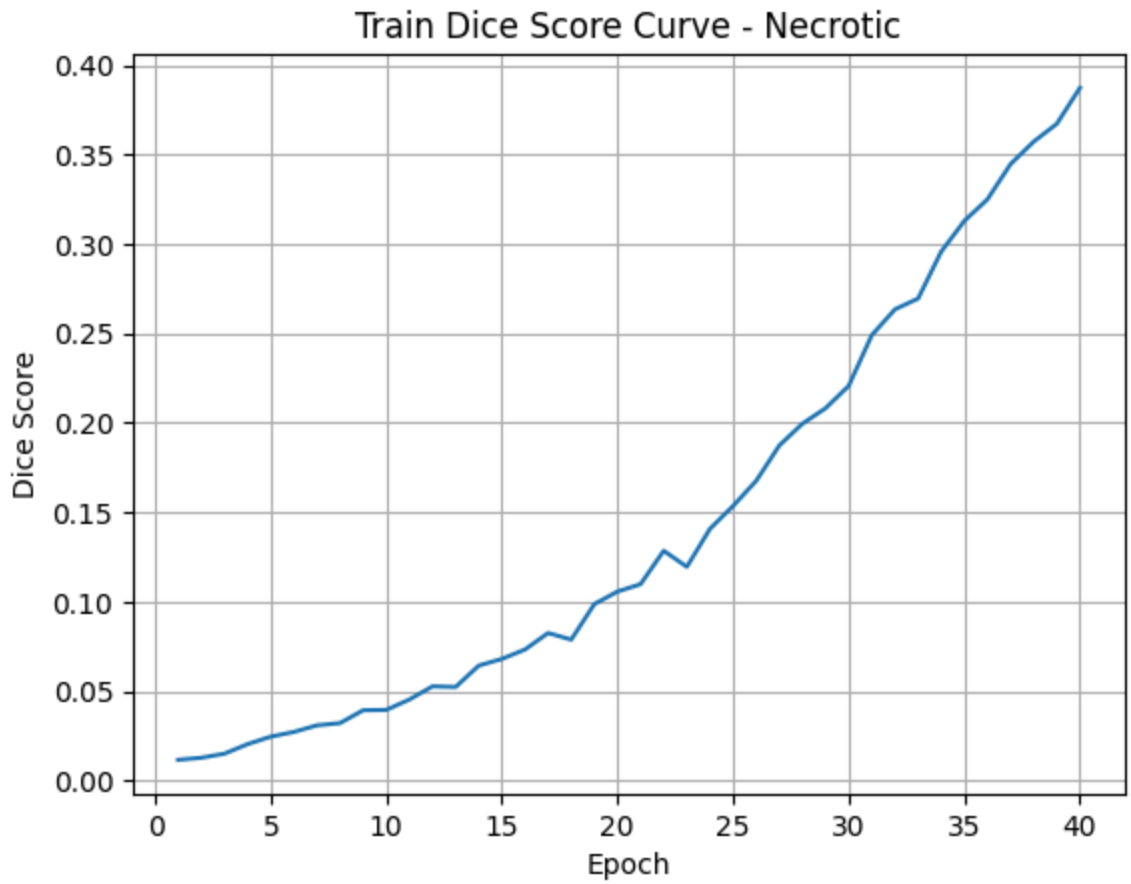
In [10]:
```
torch.save(model.state_dict(), "unet3d_2.1.pth")
del model
del train_dataloader
del test_dataloader
```

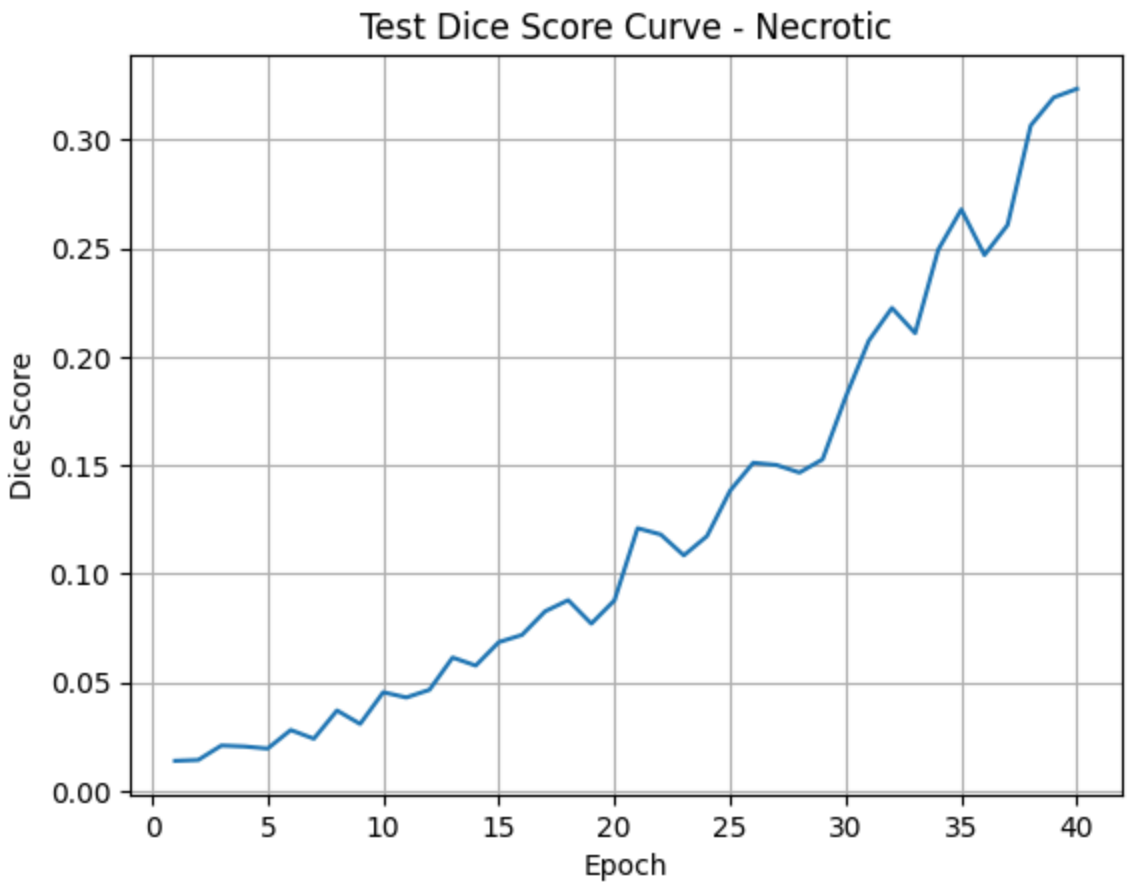
In [11]:
```
from utility.plotting import plot_graph
plot_graph(train_losses, "Train Epoch Loss Curve", "Epoch", "Loss")
plot_graph(test_losses, "Test Epoch Loss Curve", "Epoch", "Loss")
```

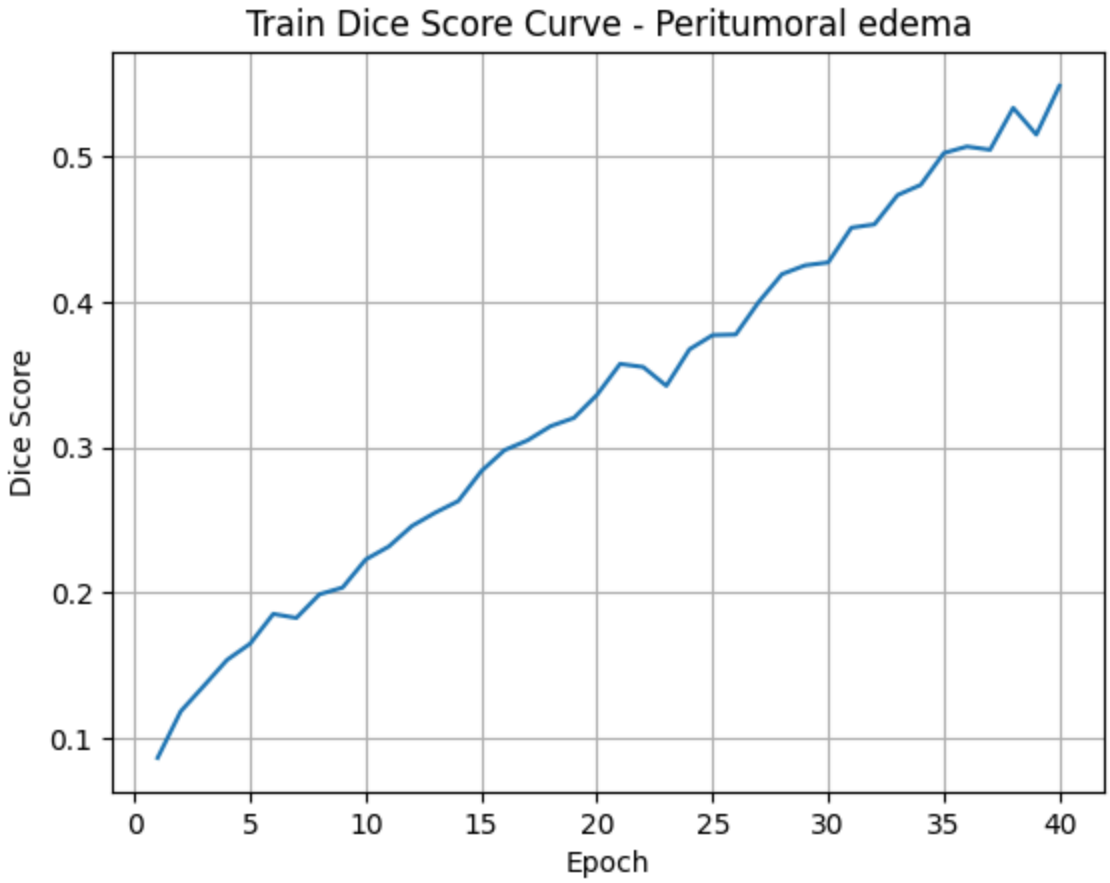

Train Epoch Loss Curve
Loss
Epoch
2.0
1.8
1.6
1.4
1.2
1.0
0.8
0
5
10
15
20
25
30
35
40

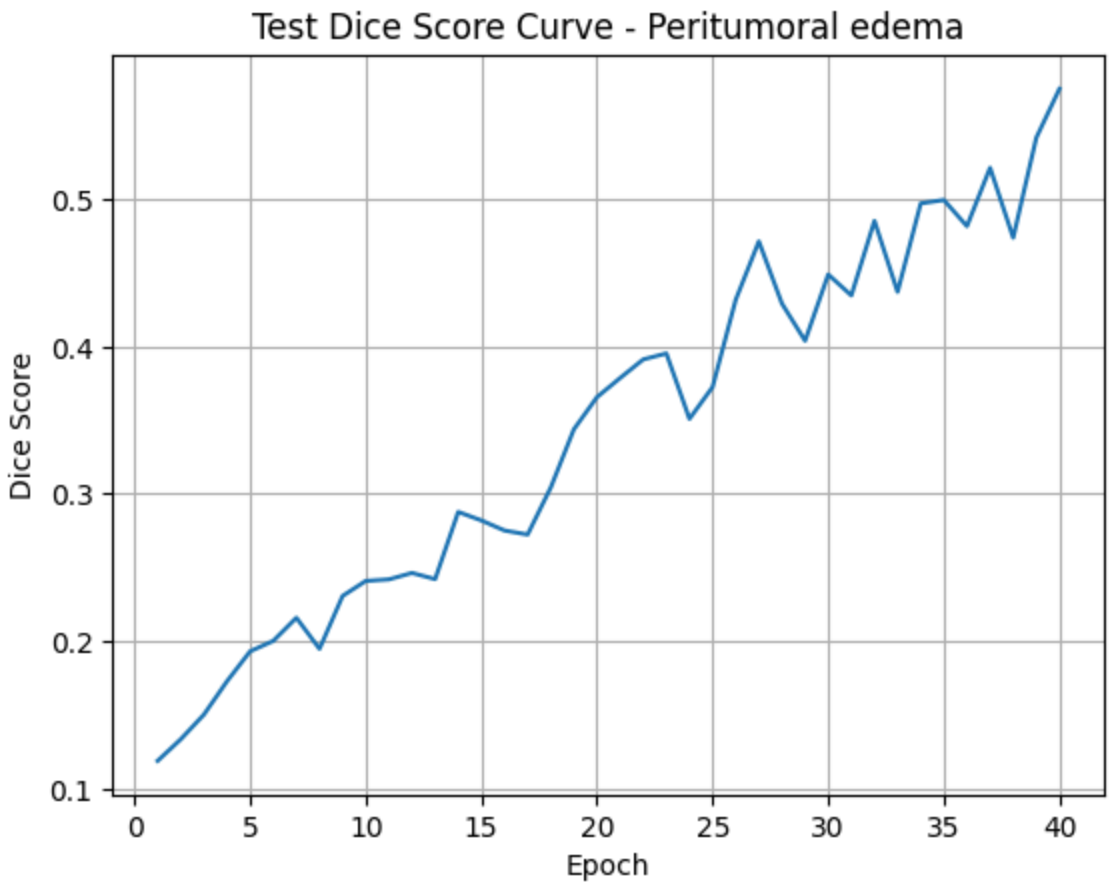

Test Epoch Loss Curve
Loss
Epoch
2.0
1.8
1.6
1.4
1.2
1.0
0
5
10
15
20
25
30
35
40

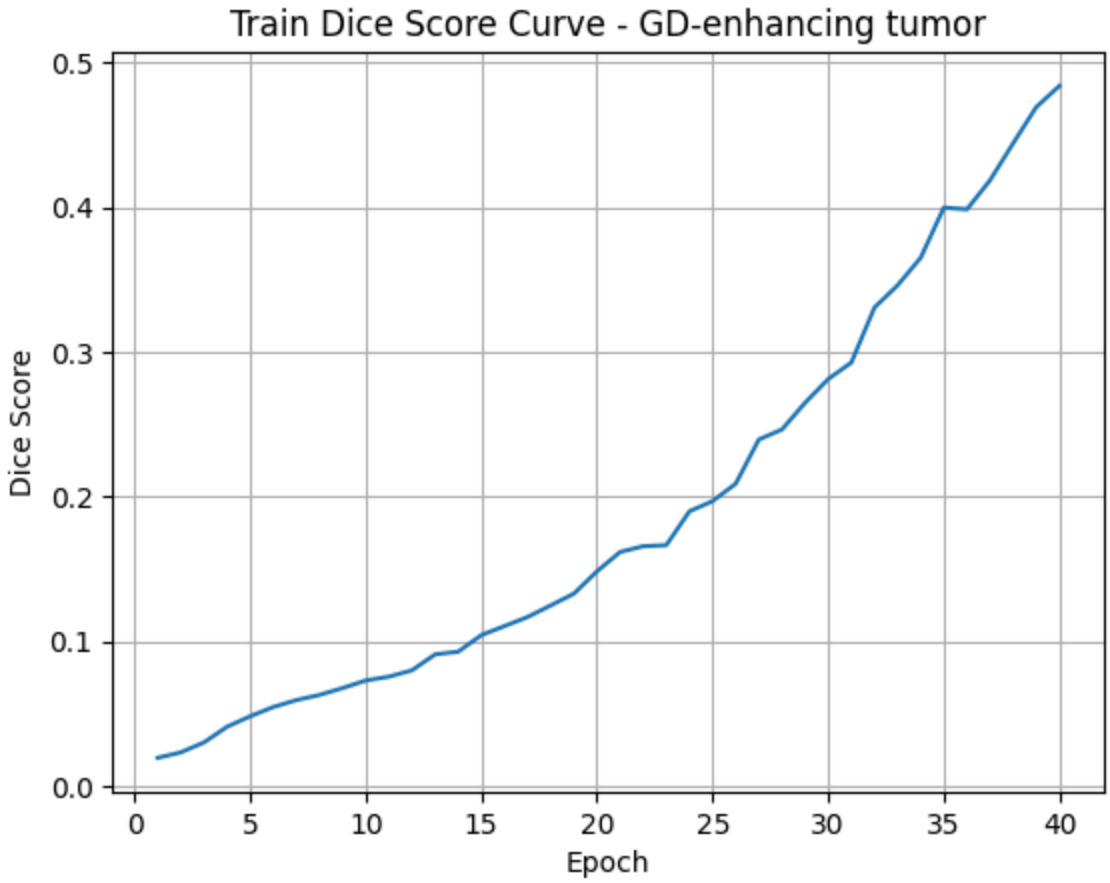
In [12]:
```
plot_graph([t.detach().cpu()[1] for t in train_dice_scores], "Train Dice Score Curv
plot_graph([t.detach().cpu()[1] for t in test_dice_scores], "Test Dice Score Curve

plot_graph([t.detach().cpu()[2] for t in train_dice_scores], "Train Dice Score Curv
plot_graph([t.detach().cpu()[2] for t in test_dice_scores], "Test Dice Score Curve

plot_graph([t.detach().cpu()[3] for t in train_dice_scores], "Train Dice Score Curv
plot_graph([t.detach().cpu()[3] for t in test_dice_scores], "Test Dice Score Curve
```

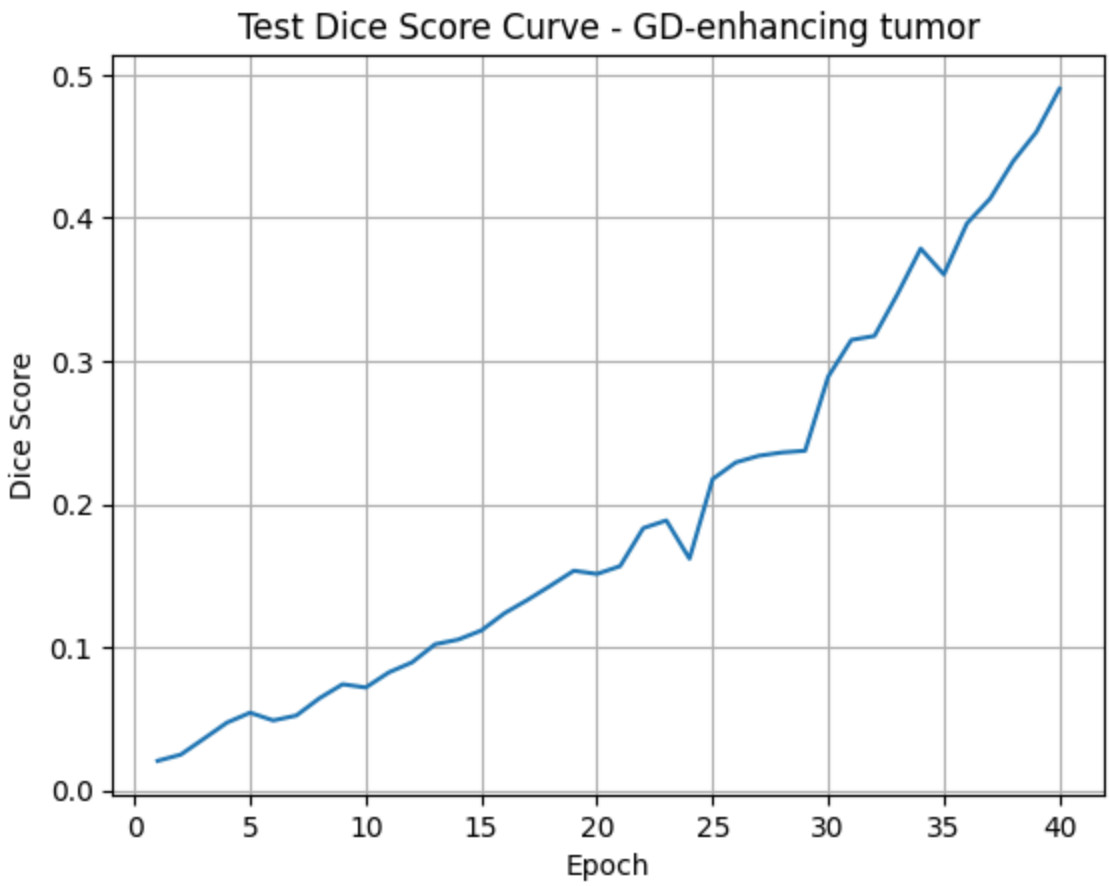



Test Dice Score Curve - Necrotic
Dice Score
0.00
0.05
0.10
0.15
0.20
0.25
0.30
0
5
10
15
20
25
30
35
40
Epoch


Train Dice Score Curve - Peritumoral edema
Dice Score
0.1
0.2
0.3
0.4
0.5
0
5
10
15
20
25
30
35
40
Epoch


Test Dice Score Curve - Peritumoral edema
Dice Score
Epoch
0.1
0.2
0.3
0.4
0.5
0
5
10
15
20
25
30
35
40


Train Dice Score Curve - GD-enhancing tumor
Dice Score
Epoch
0.0
0.1
0.2
0.3
0.4
0.5
0
5
10
15
20
25
30
35
40


Test Dice Score Curve - GD-enhancing tumor
Dice Score
Epoch
0.0
0.1
0.2
0.3
0.4
0.5
0
5
10
15
20
25
30
35
40